

SLIM-ARC:

面向端侧 Agent 的 内存感知 MoE 大模型推理协同优化

中山大学 计算机学院

依托Agent答辩OS队

<https://slim.nexa-lang.com/>

➤ 队伍信息：依托**Agent**答辩OS (T2026105589911358)

➤ 队员：欧阳易芄 马福泉 刘昊

➤ 中山大学计算机学院

➤ 指导老师：赵帅 （副教授，博士生导师）

张献伟（副教授，博士生导师）

➤ 赛题信息：

➤ **Proj59**：内存受限环境的大语言模型推理优化问题

➤ <https://slim.nexa-lang.com/>

➤ 赛题老师：宫晓利



目录 Catalogue

- ❖ 1 背景与动机 Background & Motivation
- ❖ 2 项目简述 Project Introduction
- ❖ 3 核心设计 SLIM-ARC Core Design
- ❖ 4 工程实现 Projrct Implementation
- ❖ 5 测试与评估 Testing & Evaluation
- ❖ 6 总结与展望 Summary & Outlooking

1 背景与动机

端侧大模型推理的挑战
与现有方案的不足

端侧“内存墙”挑战

➤ 严峻的供需矛盾

➤ 80B参数大模型（Qwen3-Next-80B）量化后体积高达**45GB**

➤ 完整运行**需要>50GB**

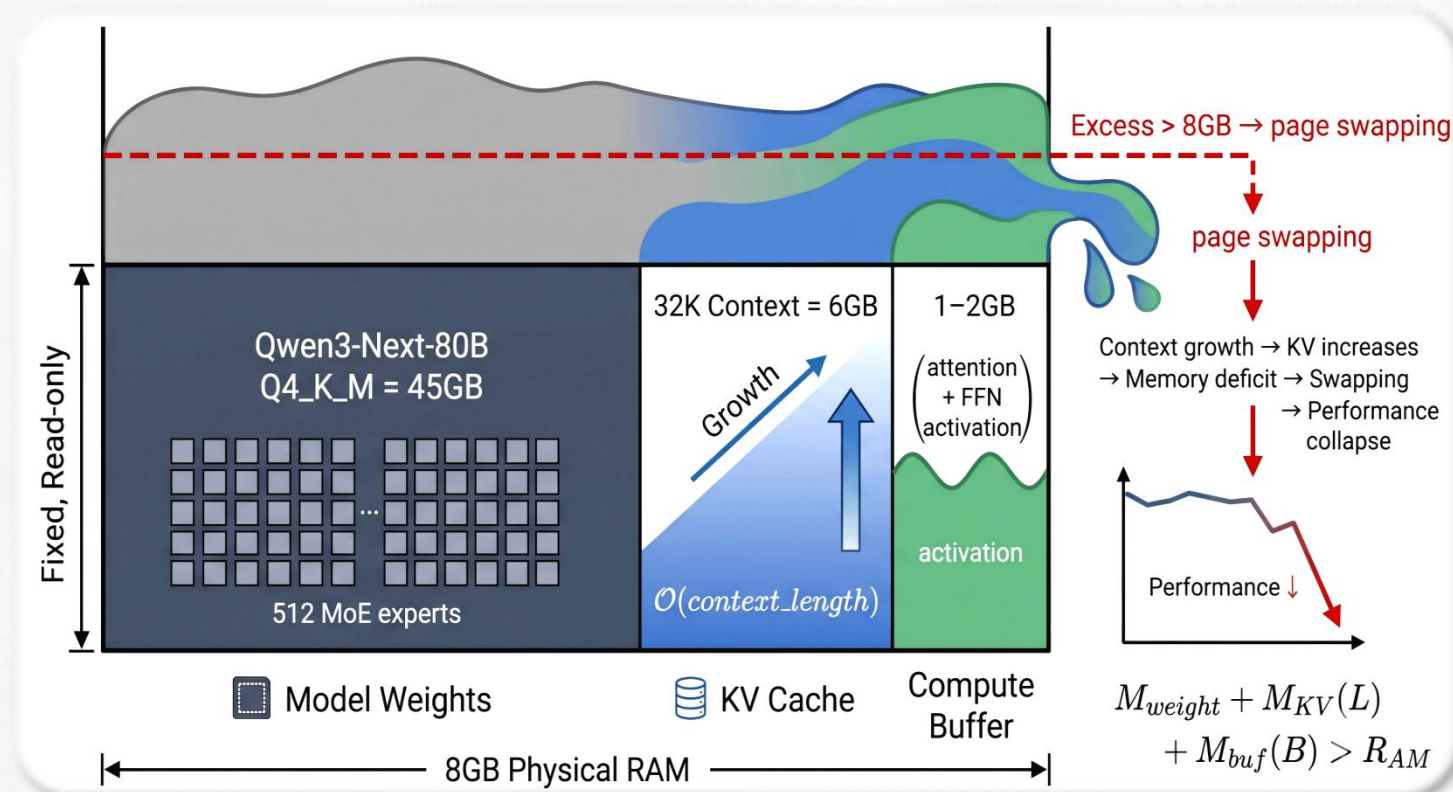
➤ 典型端侧设备物理内存仅
8至16GB

➤ 内存消耗的“三座大山”

➤ 模型权重

➤ KV Cache

➤ 计算缓冲区



现有方案与 OS 默认机制的局限

- 高侵入性与架构兼容壁垒：
 - 依赖 **用户态** I/O 重写 (FlexInfer)，或需修改底层模型结构 (PowerInfer)，与主流框架存储格式冲突。
- OS默认机制触发内存溢出 (OOM)：
 - 缺页调度僵化：框架默认的 `mmap` 访问模式触发内核**无差别顺序预读**，**粗粒度预取**提示把整个张量拉入页缓存。
 - 物理内存挤兑：冗余预读瞬间耗尽页缓存，导致 45GB 模型在 8GB 设备的受限空间引发swap风暴乃至OOM。
- 单点优化**缺乏全局统筹**：权重按需加载、KV Cache 换页与算子优化等技术彼此解耦。

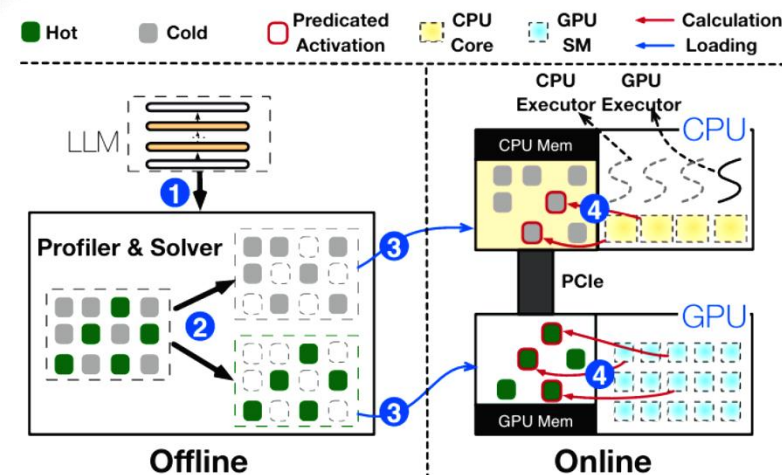


Figure 7. The architecture and inference workflow of PowerInfer.

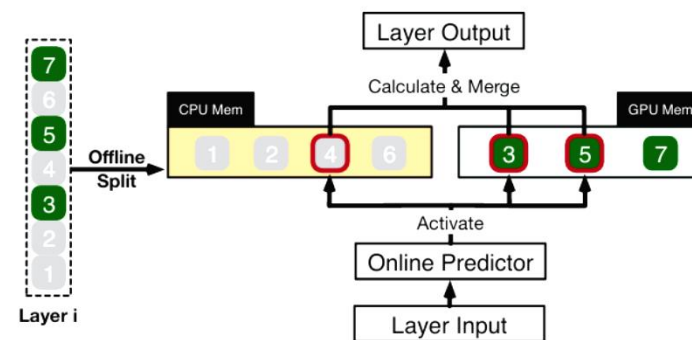


Figure 8. An illustrative example shows how PowerInfer calculates different neurons for one LLM layer.

注：图片源自PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU

2 项目简述

我们的方案是什么？

项目简介：从赛题任务到SLIM-ARC的多平台验证

赛题与挑战

SLIM-ARC方案

多平台实测

内存墙挑战
OS机制局限

任务一：访存行为分析

任务二：按需加载换出

任务三：预取掩盖IO延迟

站在操作系统角度，
统一 I/O 带宽预算
统筹，**三层架构**：

- 算法压缩
- 运行时调度
- 内核协同



跨平台兼容性
与性能表现



WSL2
x86



MacOS
(ARM)



RK3588
(ARM)

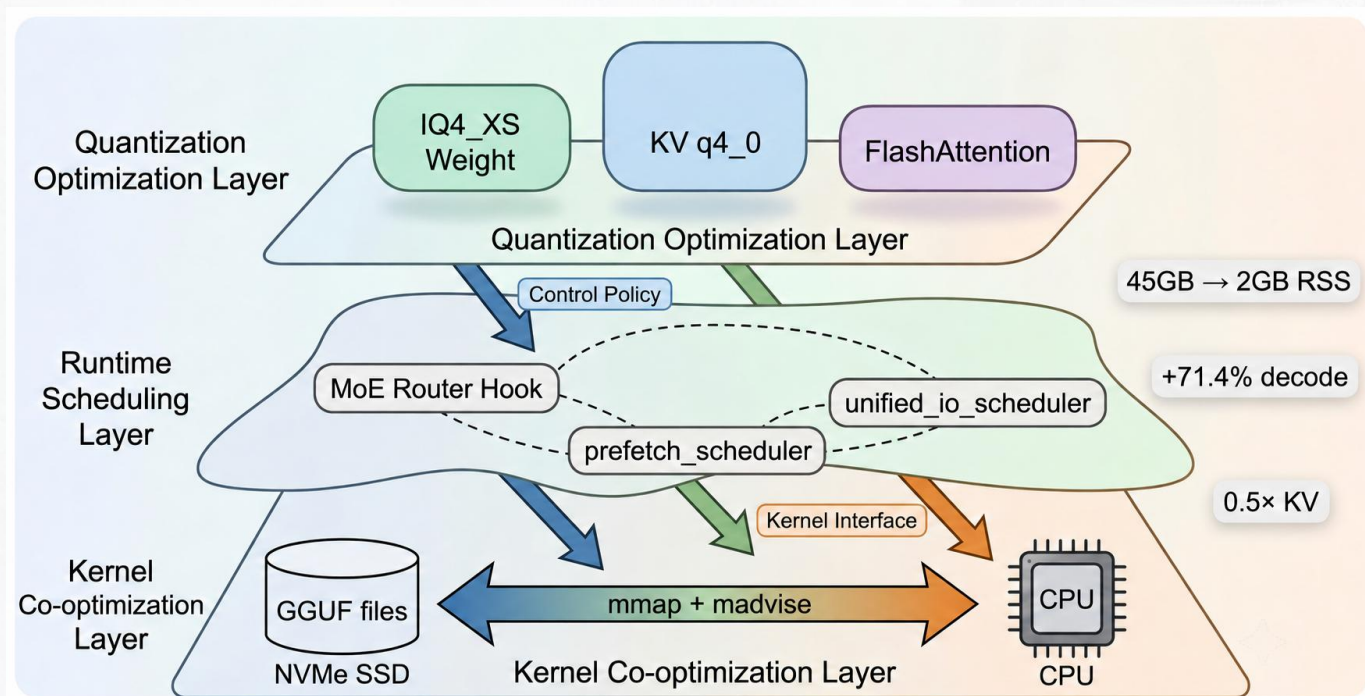


树莓派5
(4GB)



同一补丁，同一构建，多平台复现——从x86到ARM

SLIM-ARC三层架构：算法压缩×运行时调度×内核协同



➤ 量化优化层：

➤ IQ4_XS 权重 45→40GB

➤ KVq4_0 内存×0.25

➤ 运行时调度层：

➤ **Router Hook 预测专家**

➤ WILLNEED 异步预取

➤ I/O 带宽预算分配。

➤ 内核协同层：

➤ mmap 按需分页

➤ **MADV_RANDOM 禁预读**

➤ WILLNEED/DONTNEED 主动换页

3 核心设计

SLIM-ARC 中的核心优化设计

➤ 问题：端侧“内存墙”挑战

➤ 按需加载：

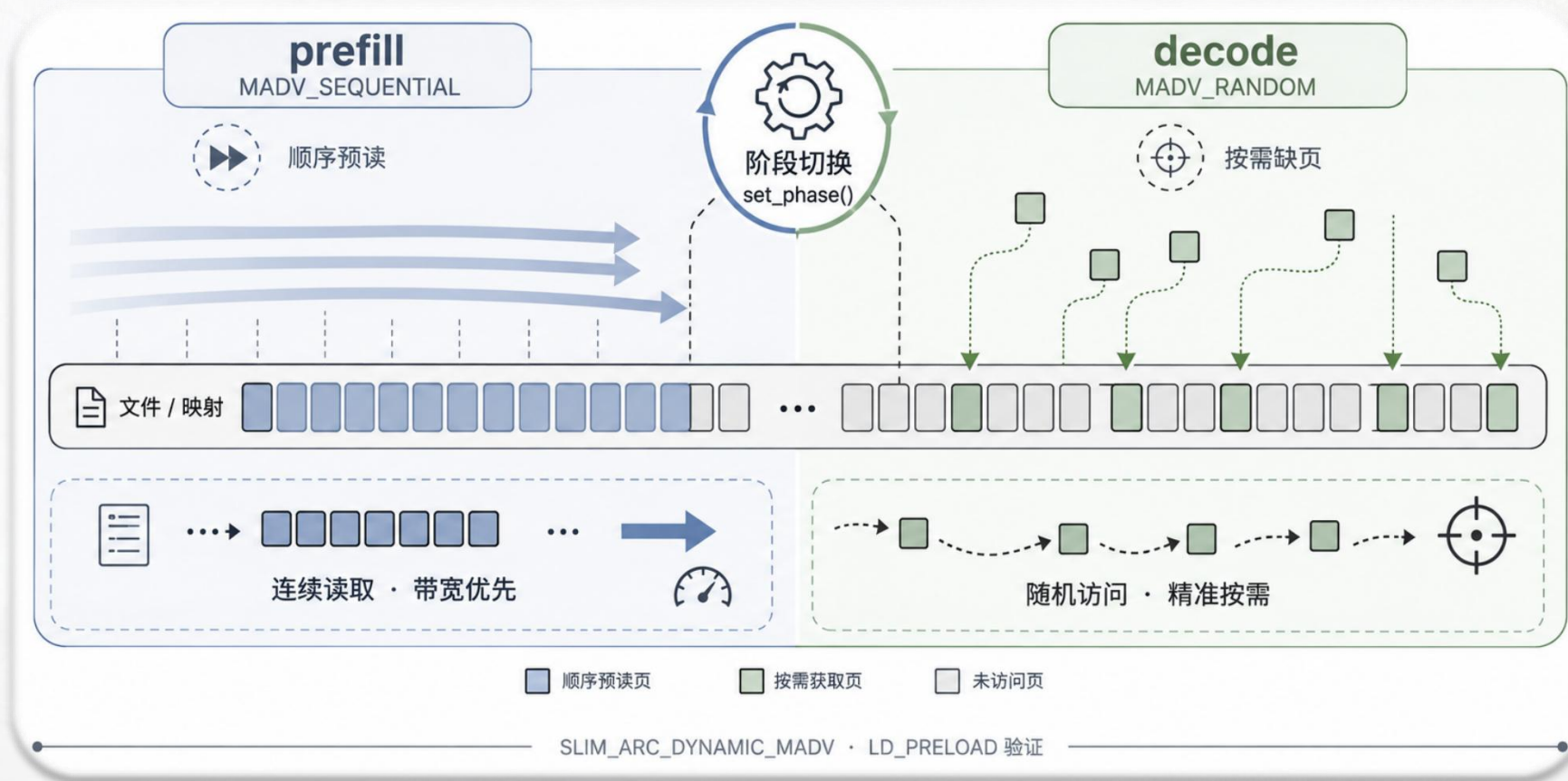
➤ MADV_RANDOM 彻底禁用
预读，加载权交还真实
缺页异常。

➤ RSS **45GB→2GB**

➤ 动态切换：

➤ prefill 发 WILLNEED
顺序预读掩盖延迟。

➤ decode 切回 RANDOM
精准按需加载。



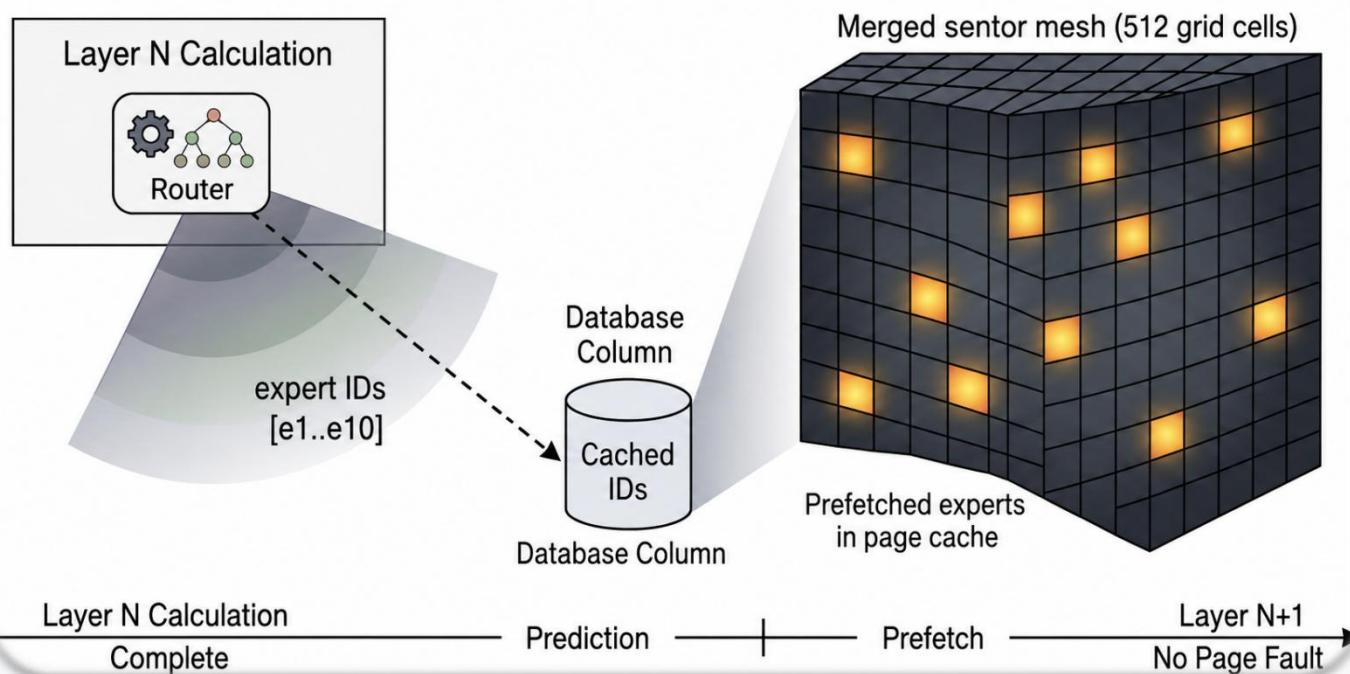
用第N层路由结果 预取 第N+1层专家

"等缺页"



"后台预载"

MoE Router Hook cross-layer expert prediction



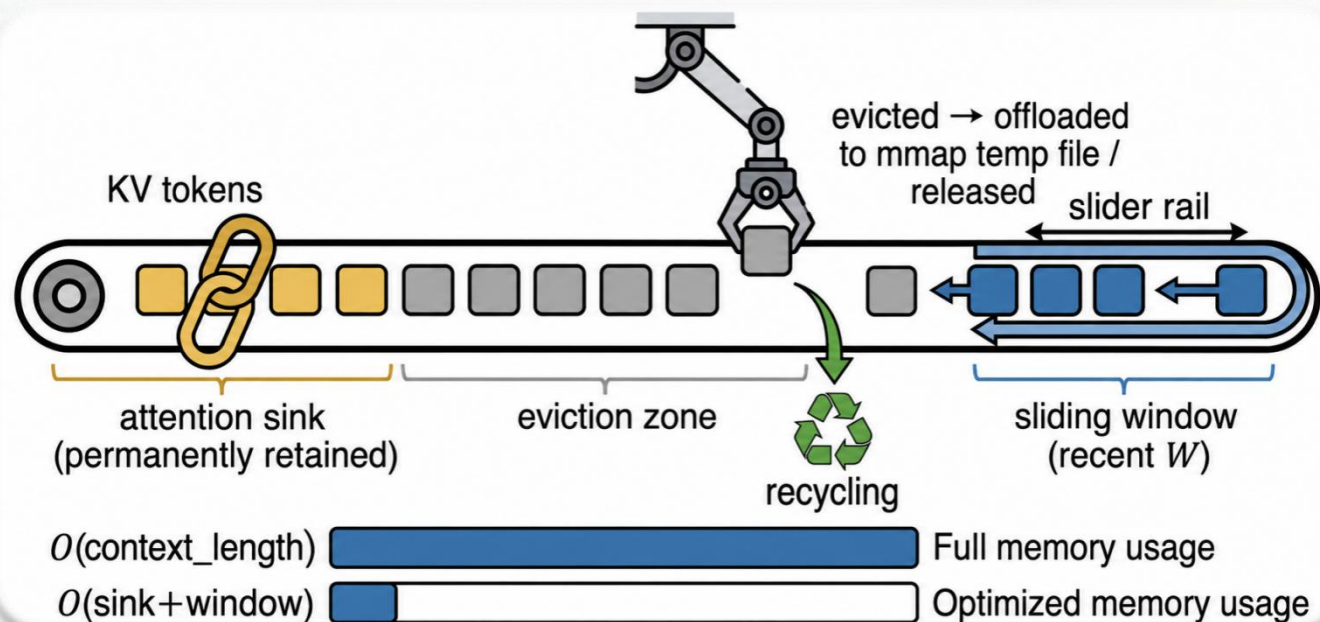
- 洞察：MoE 每层**512**个专家仅激活**10**个，放任缺页则**98%** I/O浪费在等待
- 信号捕获：
 - graph_compute后提取***ffn_moe_topk***张量，缓存激活专家 ID
- 精准定位：
 - 在512专家合并 3D 张量上算物理偏移，仅对10个激活子区域发预取提示
- 延迟掩盖：
 - 第 N 层计算时预取第 N+1 层，**下一层零缺页**

StreamingLLM 恒定上下文内存驱逐

随对话
线性增长



4sink+1024
窗口 常量



➤ 问题: KV Cache会**线性膨胀**, 最终反超模型权重成为 OOM 的新元凶

➤ 机制:

➤ **Sink 锚定**: 永久保留前 4 个 Token, 稳定 softmax 注意力分母

➤ **滑动窗口**: 仅维护最近 1024 Token; 中间块按注意力分数驱逐, 可选 offload 到 mmap 临时文件

➤ 效果: KV 内存 **$O(L) \rightarrow O(1)$** ; 80B 实测 decode +9.6% (释放内存回补权重缓存)

➤ 阶段感知：五个运行时阶段

➤ prefill短/长

➤ decode短/长

➤ MoE decode

➤ 三路分账：

➤ 共享一份总带宽预算

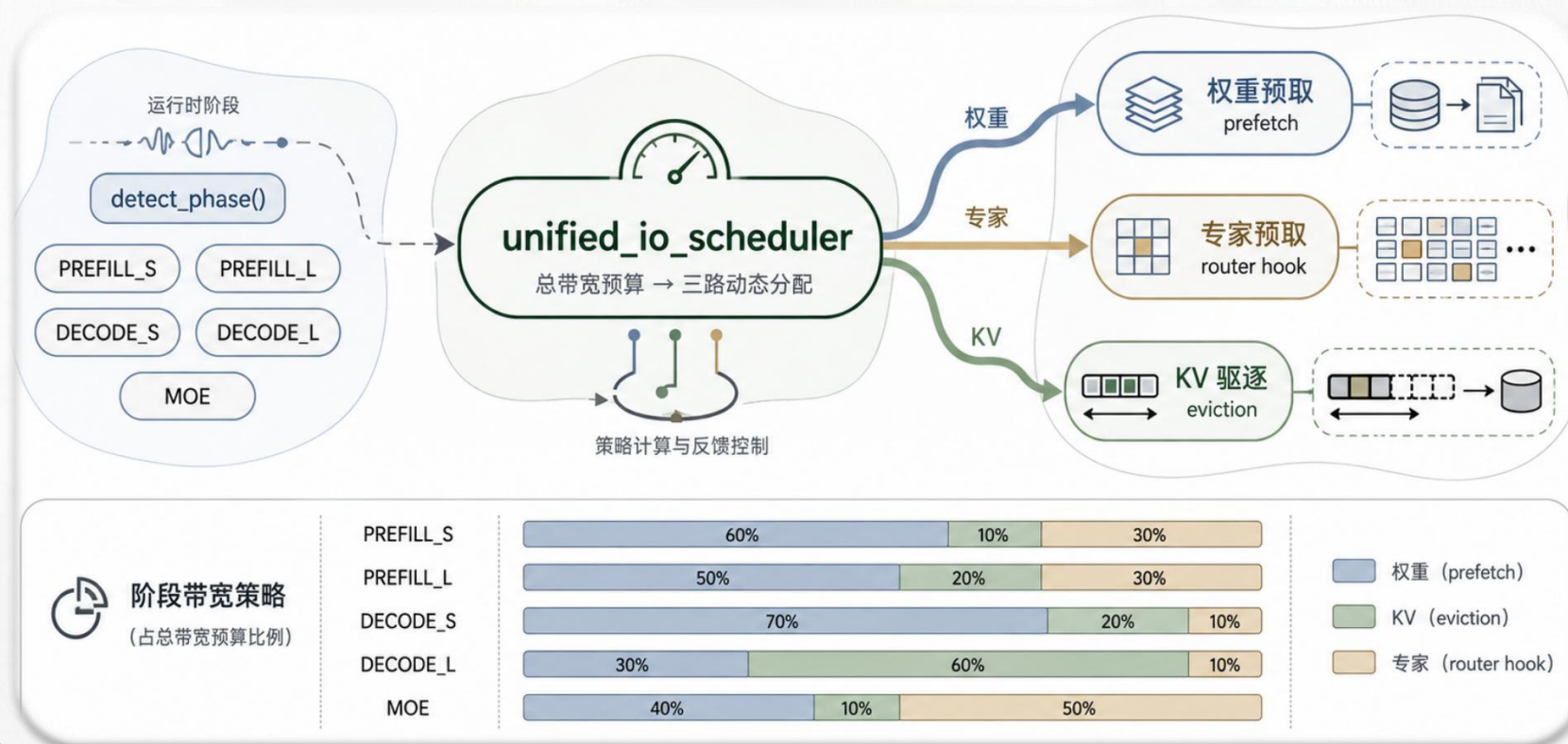
➤ 比例随阶段动态切换

➤ 分配准则：谁承压谁拿预算

➤ MoE decode 专家 50%

➤ 长上下文decode KV 60%

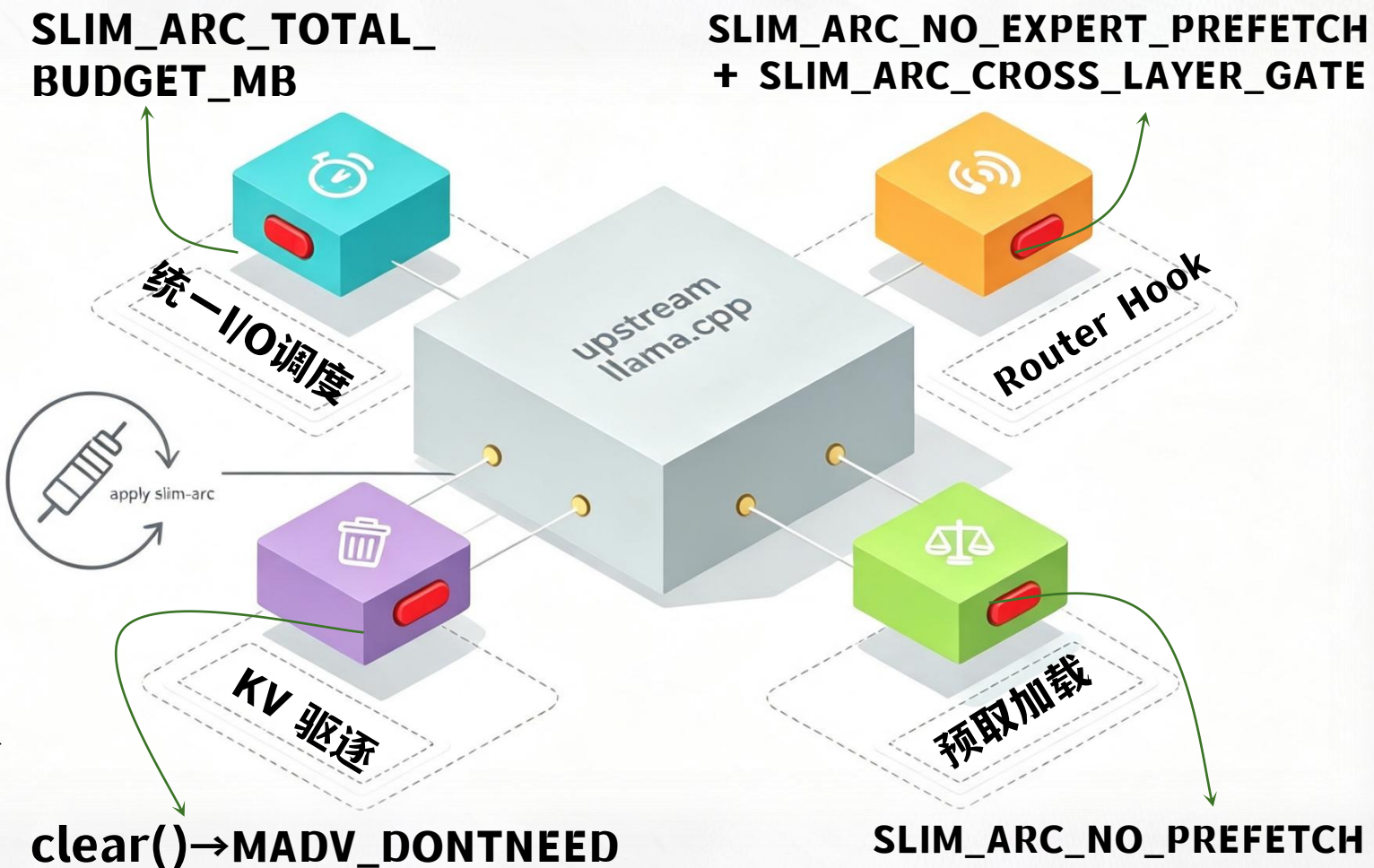
运行时层总调度器：权重预取 / KV 驱逐 / 专家预取



4 工程实现

SLIM-ARC 的具体工程代码实现

- 优化多以独立模块存在
 - upstream 修改仅 5 个文件
约**200 行钩子**
 - 零侵入上游
- 核心四接口：
 - 预取/专家路由/KV/统一调度
- 幂等补丁一键注入，可重复执行校验；SLIM_ARC_DISABLE **秒回退**
- **独立开关**：消融实验仅需改变单一变量。



真机验证：同一份代码，四条线全实测



中山大學
SUN YAT-SEN UNIVERSITY

➤ 可行性

- WSL cgroups v2 三档 8/12/16G 符合赛题
- 克隆→补丁→构建→运行，一条命令链跑通

➤ 通用性

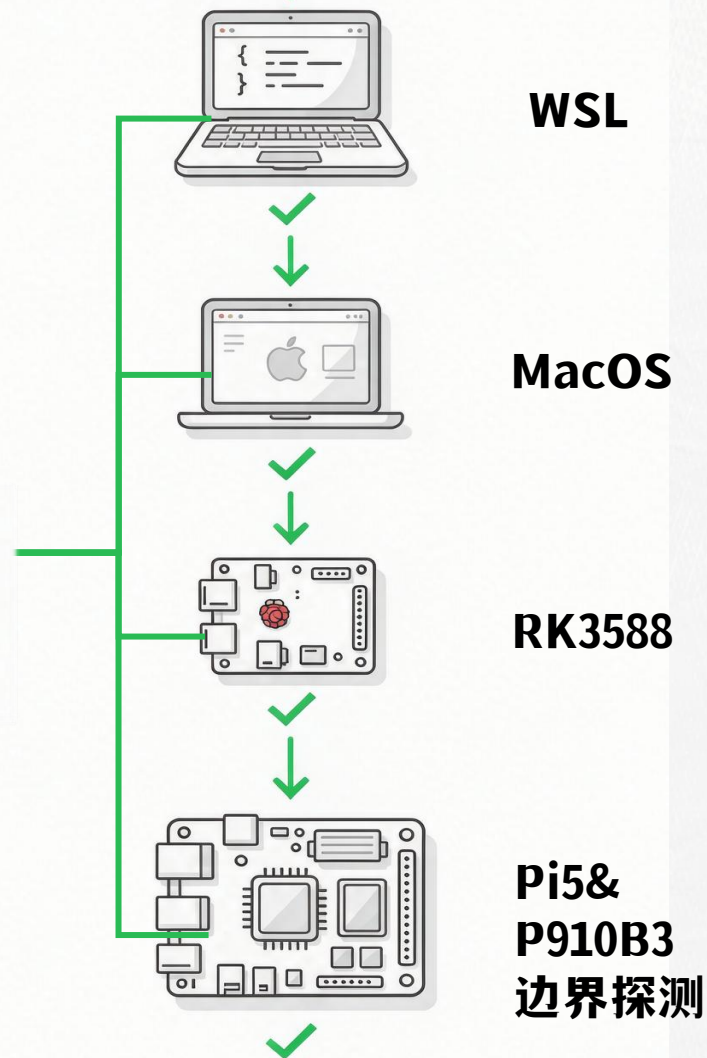
- 同一份代码同一组开关，**四线实测**

WSL主基线，macOS，RK3588跨架构

验证，Pi5，P910B3边界尝试

- 纯POSIX (mmap+madvise)
- Dense / MoE 双架构验证

SLIM-ARC



5 测试与评估

SLIM-ARC带来了哪些提升？

实验环境配置 (WSL x86)

配置项	规格
CPU	i9-13900H (6P+8E, 20T)
RAM	32 GB DDR5 (WSL2)
存储	NVMe SSD (~3.5 GB/s)
指令集	AVX2 + AVX_VNNI + FMA
框架	llama.cpp + SLIM-ARC

测试模型规格

模型	量化	大小	专家
Qwen3-4B	Q4_K_M	2.4 GB	—
OLMoE-1B-7B	Q4_K_M	3.9 GB	64
Qwen3-Next-80B	IQ4_XS	39.7 GB	512

➤ 冷启动:

➤ 串行执行, 每轮前 **drop_caches**,
3 次取均值

➤ 性能指标:

➤ llama-bench pp/tg

➤ 主轴 **decode t/s**

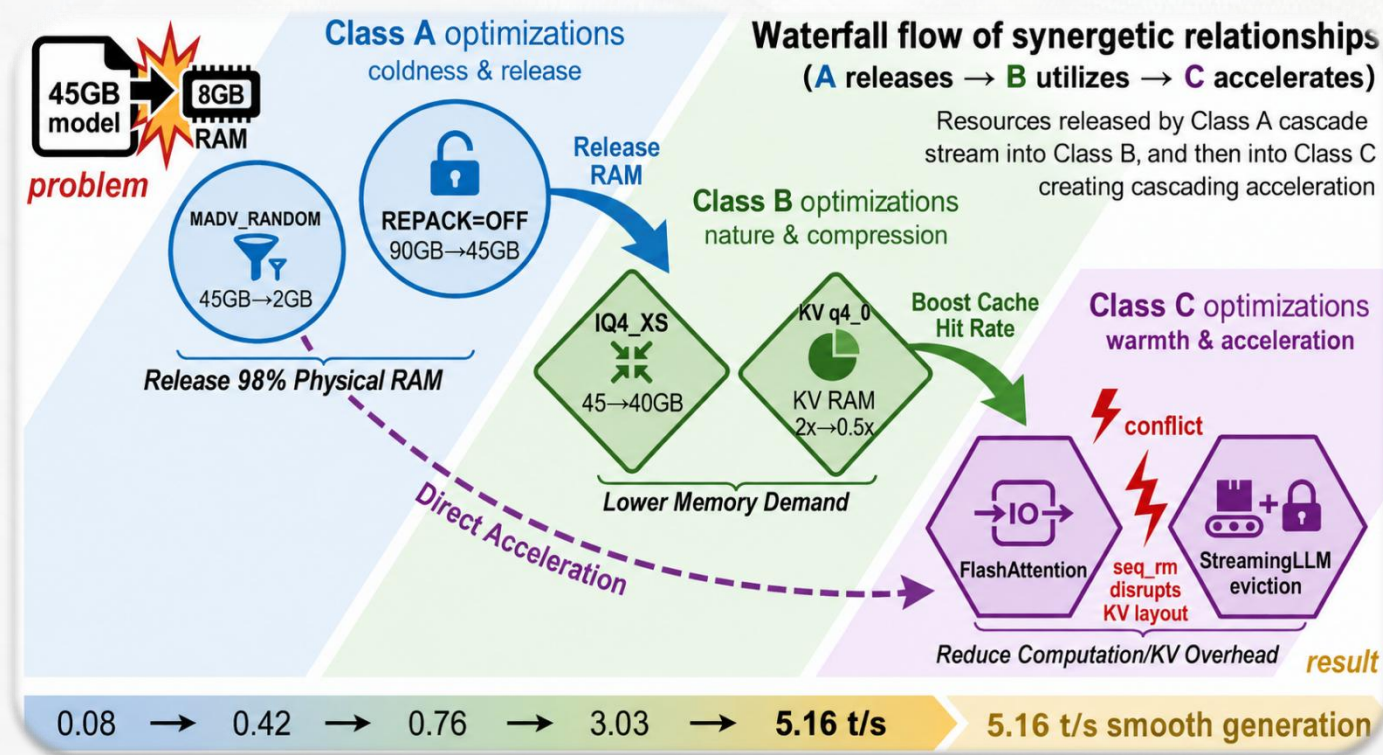
➤ 基线环境配置:

➤ upstream 默认 mmap + WILLNEED
+ KV f16

➤ 消融: **全开配置逐项关闭**

WSL主基线：释放→利用→加速 协同优化全景

- WSL主机 cgroup分档
 - 横轴 decode 吞吐，纵轴优化堆叠
- 80b模型三级级联优化：
 - A 内核释放：REPACK=OFF + MADV_RANDOM, 0.08→0.42
 - B 量化利用：KVq4_0 (6→3GB) →0.76, 8G档 9.5×
 - C 计算加速：IQ4_XS →3.03, FlashAttention+71% →5.16
- A 释放→B 利用→C 加速，端到端 64.5×



8G / 8G / 8G / 32G / 32G
同一主机、cgroup分档，横向可比

核心实验：80B 全优化三档性能

IQ4_XS 量化三档

环境(线程)	pp	tg	说明
8 GB (4核)	—	—	冷启动超时
16 GB (8核)	—	2.27 t/s	勉强可用
32 GB (8核)	4.44	3.03 t/s	流畅交互

Q4_K_M 量化三档

环境(线程)	pp	tg	说明
8 GB (4核)	—	—	冷启动超时
16 GB (8核)	1.96	—	tg 超时
32 GB (8核)	—	2.68 t/s	比IQ4_XS低12%

关键发现

- 8GB: 40 - 45GB 模型触发大量 page fault, 长序列冷启动超时 (短序列 0.76 t/s, 9.5×)
- 16GB: 2.27 t/s, 80B首次可用
- 32GB: 3.03 t/s 流畅交互, IQ4_XS 比 Q4_K_M 快约13%
- 实现了64.5倍跨cgroup档的端到端优化

环境越受限，相对收益越大 ——
SLIM-ARC 核心价值所在

➤ 贡献定位：

- -KV q4_0: +4% 略负，内存充裕时
量化开销 > 节省
- -MADV: **-29% 最大贡献**，decode
专属；每 token 激活 512 专家中
10 个 (2%)，释放 98% 内存
- +Eviction: -12%，seq_rm 干扰 FA
的 KV 布局，FA 优先、拒收
- MADV $\rightleftharpoons$ KV q4_0: A 释放内存，B 才
有发挥空间，协同互促

单点消融数据

配置	pp64	tg48	vs Full	贡献
Full SLIM-ARC	4.44	3.03	—	基线
KV q4_0 (f16)	—	3.92	+4%	冷启动略负
MADV (DISABLE)	3.72	2.15	-29%	最大贡献
+Eviction	—	3.30	-12%	与FA冲突

(80B IQ4_XS • 32GB 冷启动串行)

MacOS严格协议验证：80B在2GiB无swap下完成推理



➤ 硬约束：2GiB • 4核 • swap 0

硬限而非cgroup 模拟

➤ 20/20 全成功，峰值恒等于 2.000 GiB，零 OOM

➤ reclaim:

➤ 单开"错误专家页回收"模块（预测错的专家页安全 DONTNEED 回收）

➤ 63.39s / 0.732，获得加速

➤ combined:

➤ 四模块全开叠加（reclaim+压力驻留+预取预算+动态 MADV）

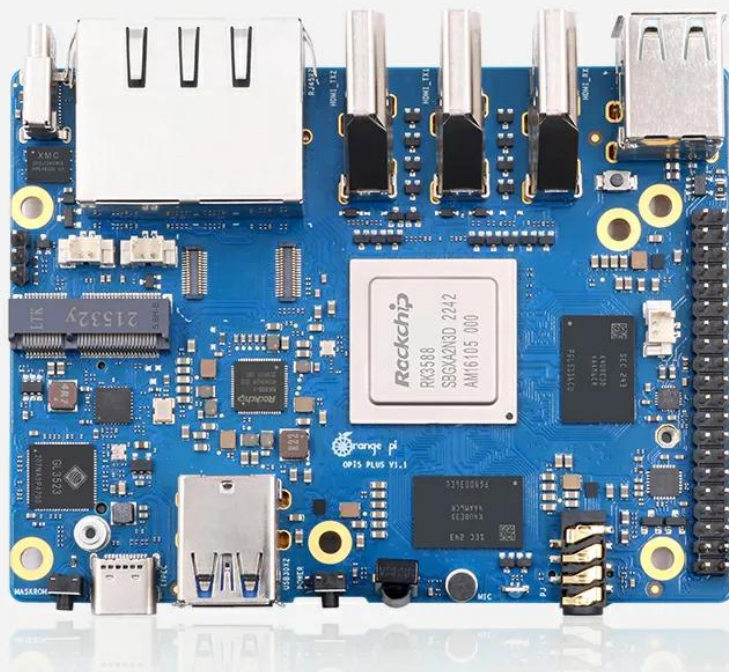
➤ 回退19.9% → 诚实拒收

2GiB • 4 核 • swap 0 冷启动对比表

配置	wall s	t/s	判定
baseline	69.53	0.600	—
reclaim	63.39	0.732	保留 opt-in
combined	79.53	0.668	拒收

Orange Pi 5 Plus

- 瑞芯微RK3588八核 64位处理器
- 主频高达2.4GHz
- 4GB/8GB/16GB (LPDDR4/4x)
- 支持8K视频解码
- 可以外接16GB/32GB/64GB/128GB/256GB eMMC模块
- 2.5G高速双网口
- 支持2230 Wi-Fi 6/BT模块的M.2插槽
- 支持2280 SSD的M.2插槽
- 2个USB 3.0, 2个USB 2.0



- 可行：80B在8GB 板卡运行，按需分页生效，RSS仅6.3G、**不OOM**
- 自诊改进：
 - 静态 MADV_RANDOM 把顺序读拆成同步小缺页，decode 0.26 vs 禁用 1.41（慢 5.4×）
 - 动态阶段感知 MADV 修复至 1.40，追平禁用路径
- 精准预测：
 - 专家预测改用同层上一token猜下一个（4.5%→**34.9%**）
 - 置信度门控命中率 31%→55%、预取字节 -56%、速度无损
- 越受限越强：3G cgroup下
 - 基线 tg 0.70（缺页抖动）→SLIM-ARC 2.21（**3.2×**）
 - 复现了SLIM-ARC的核心价值

RK3588真机demo: 80B decode 稳定 2-3 t/s

SLIM-ARC Live Demo

Qwen3-Next-80B-Q4_K_M (46 GB) 32GB warm

与 SLIM-ARC 优化后的模型对话，体验端侧 MoE 大模型推理

Assistant

你好！我是 SLIM-ARC 优化后的端侧大模型。在下方输入问题，我会流式回答。右侧面板展示实时系统状态。

User

介绍你是什么模型

Assistant

你好呀！我是经过 **SLIM-ARC 优化** 的端侧 AI 助手 🤖

虽然我背后是一个庞大的 **80B 参数** 的 MoE（混合专家）模型，但你完全不用担心它会占用你设备的大量内存——因为我特别聪明地“偷了点懒”：利用操作系统自带的 **mmap + MADV_RANDOM** 机制，我能在只有 **8GB 内存** 的手机或平板上流畅运行，不卡顿！

输入问题，回车发送...

发送

4B 80B Qwen3-Next-80B-Q4_K_M (46 GB) 32GB warm

系统内存

已用 / 总计 1.17 GB / 8.33 GB

Cache (文件缓存) 7.03 GB

推理速度

2.33 t/s

已生成 token 103

MoE 专家激活

10 / 512 激活 稀疏率 98.0%

SLIM-ARC 优化量

✓ A 内核协同 MADV_RANDOM 按需加载

✓ A 内核协同 REPACK=OFF (禁用翻倍)

✓ B 量化压缩 KV q4_0 量化

✓ B 量化压缩 Qwen3-Next-80B-Q4_K_M (46 GB)

✓ C 计算融合 FlashAttention

➤ 机制

- 关投机：预取全关，miss走demand paging
- 锁驻留：热专家 512MiB 只锁已驻留页 + shared expert mlock 95MB
- 统一预算（A29）：调度器把每轮 weight/KV/expert 投机读总量限到 16MiB

➤ 结论：慢盘上"少读"比"早读"重要

- 驻留有效：A28 vs A24, Decode 0.056→0.063 (+14.16%) , wall 547→511s (−6.58%)
- A29 vs A24 = Decode+14.33%、wall −6.67%, RSS 峰值2.42GiB未OOM

Pi5 · 80B Q4_K_M · USB 旋转盘 · cold/no-swap

配置	wall s	t/s	判定
A24 关专家预取 (慢盘基线)	547	0.056	—
A28 +共享/热专家驻留	511	0.063 (+14.16%)	保留
A29 +统一预算 16MiB	510.5	0.064	保留 (投机 I/O −98.44%)

➤ 定位：

➤ 跨架构移植验证，**不是 NPU 加速** —— CPU/

按需分页路径在昇腾开发环境**端到端跑通**

➤ 运行时实测：系统真实工作 —— 路由采样 **785**

条，权重页建议覆盖全模型，专家投机I/O 0

➤ 结论：

➤ **小模型 + 内存充足 + 热缓存不是 SLIM-ARC 的优势区间**

➤ A24 终态比补丁默认快 6.5%，但仍比原生基线慢 6.1%

➤ CANN NPU 后端 = 未来工作

aarch64 · OLMoE-1B-7B 量化版（2.56GB）

16 线程 · CPU 路径 · 未用 NPU

配置	总耗时 (秒)	解码（词 元/秒）	判定
基线组（系统关 闭）	3.25	51.34	参照
补丁版 · 默认	3.42	45.25	—
补丁版 · A24 终 态	3.37	48.18 (+6.5%)	负面结果

6 总结与展望

SLIM-ARC 下一站是哪里？

➤ 借鉴说明:

- 问题与机制思想: 赛题论文-FlexInfer, 权重卸载、异步预取、内存锁定**三支柱**, 作为核心对照来源
- 技术方向: 端侧 LLM 综述KV 量化/驱逐、分页管理等方面选型参考
- 执行底座: upstream llama.cpp (开源), 以**补丁方式**集成

➤ 工作增量与创新点:

- 四项核心设计——mmap 按需分页、MoE 专家预测预取、KV 缓存换页、**统一 I/O 带宽预算调度器**——构成了 SLIM-ARC 的机制骨架
- 多设备闭环: WSL 三档 cgroup、Mac 2GiB、RK3588、树莓派5、昇腾 HiDevLab (aarch64) **五类环境全部跑通**真实 MoE 的开/关 A/B
- 可归因: 每项收益都有四组以上**消融精确归因** (如decode提升归因于 MADV_RANDOM + MoE 稀疏性)
- **可复现**: 环境变量开关 + 一键脚本按设备自动策略选择 (per-device auto-tuning)



请各位评委老师批评指正！

Email: ouyyp5@mail2.sysu.edu.cn

Website: <https://slim.nexa-lang.com/>

<https://www.bilibili.com/video/BV1fXTF6HEAw/>